

Universidade Estadual de Londrina

Play UUel

Ulrich Daniel

Londrina

30 de setembro de 2026

Sumário

1	Introdução	2
1.1	Desenvolvimento do Jogo Duelo de Reflexo	3
1.2	Desenvolvimento do Jogo da Memória	7
1.3	Desenvolvimento do Jogo da Velha	10
1.4	Desenvolvimento do Jogo Pedra, Papel e Tesoura	12
1.5	Desenvolvimento do Jogo da Força	15
2	Resultados Obtidos	18

1 Introdução

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

1.1 Desenvolvimento do Jogo Duelo de Reflexo

O jogo Duelo de Reflexo foi desenvolvido utilizando HTML, CSS e JavaScript e permite a participação simultânea de dois jogadores. O objetivo consiste em pressionar a tecla correspondente ao jogador o mais rapidamente possível após a exibição do sinal de início da rodada.

Antes de cada rodada, o sistema aguarda um intervalo de tempo aleatório. Caso um jogador pressione sua tecla antes do sinal, a jogada é considerada inválida e o participante é penalizado por ter quebrado as regras. Após o início válido, o programa registra o tempo de reação e determina o dano causado ao adversário.

A estrutura HTML foi utilizada para organizar os elementos da interface, incluindo o cenário da luta, os personagens, as barras de vida, as mensagens de orientação e os controles da partida.

```
1 <!DOCTYPE html>
2 <html lang="pt-br">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>DUEL</title>
7   <link rel="stylesheet" href="reflexo.css">
8 </head>
9 <body>
10  <a href="../../index.html" class="back-link">- MENU</a>
11  <h1>Duelo de Reflexo</h1>
12
13  <div id="arena">
14
15    <div class="hudVida">
16
17      <div class="vidaFundo">
18        <div id="vidaPreenchimento1"></div>
19      </div>
20
21      <div class="vidaFundo2">
22        <div id="vidaPreenchimento2"></div>
23      </div>
24
25      
26      
27
28    </div>
29
30    
31    <div id="tecla">?</div>
32    <div id="resultado"></div>
33    
34
35  </div>
36
37  <div class="controles">
38    <button id="iniciar">Iniciar rodada</button>
39    <button id="reiniciar">Reiniciar jogo</button>
40  </div>
41  <p id="mensagem">Clique para começar.</p>
42
43  <script src="reflexo.js"></script>
44 </body>
45 </html>
```

Figura 1: Estrutura HTML utilizada no jogo Duelo de Reflexo.

A estilização visual foi implementada utilizando CSS, responsável pelo posicionamento dos

personagens, organização das barras de vida, composição do cenário e aplicação dos efeitos visuais apresentados durante os ataques.

```
7 #resultado
8 {
9   position: absolute;
10  left: 50%;
11  bottom: 20px;
12
13  transform: translateX(-50%);
14
15  font-weight: bold;
16  font-size: 40px;
17  color: ■white;
18 }
19
20 #vidas
21 {
22   position: absolute;
23   top: 20px;
24   width: 100%;
25
26   display: flex;
27   justify-content: space-between;
28
29   color: ■white;
30   font-size: 30px;
31 }
32
33 #player1, #player2
34 {
35   transition: transform 0.15s ease;
36 }
37
38 #player1.atacando
39 {
40   transform: translateX(200px);
41 }
42
43 #player2.atacando
44 {
45   transform: scaleX(-1) translateX(200px);
46 }
47
48 .hudVida
49 {
50   position: relative;
51   width: 170px;
52   height: 700px;
```

Figura 2: Trecho da estilização CSS utilizada no jogo Duelo de Reflexo.

A lógica da aplicação foi implementada em JavaScript. O programa controla o intervalo aleatório entre as rodadas, identifica o jogador que pressionou a tecla primeiro, calcula o tempo de reação e determina o dano causado.

O dano aplicado varia de acordo com a velocidade da resposta. Tempos menores resultam em ataques mais fortes, enquanto respostas mais lentas causam uma quantidade menor de dano. O código também atualiza as barras de vida, controla o encerramento da partida e exibe o jogador vencedor.

A Figura 4 apresenta a tela do jogo após o encerramento de uma partida.

```

152 document.addEventListener("keydown", function(event)
153 {
154     if (event.key === teclaAtual1 && jogoAtivo === true)
155     {
156         jogoAtivo = false;
157
158         const tempoReacao = Date.now() - tempoInicio;
159         let dano = 1;
160
161         if (tempoReacao < 350)
162         {
163             dano = 3;
164         }
165         else if (tempoReacao < 450)
166         {
167             dano = 2;
168         }
169
170         vida2 -= dano;
171         atualizarBarras();
172         mostrarEfeito(dano, tempoReacao);
173         player1.src = "imgsReflexo/macacoGolpe.png";
174         player2.src = "imgsReflexo/macacoDano.png";
175
176         if (vida2 <= 0)
177         {
178             atualizarBarras();
179             player1.classList.add("atacando");
180
181             resultado.innerText = "PLAYER 1 VENCEU!";
182             tecla.innerText = "";
183
184             player1.src = "imgsReflexo/macacoVitorioso.png";
185             player2.src = "imgsReflexo/macacoDerrotado.png";
186
187             jogoEncerrado = true;
188             return;
189         }
190
191         setTimeout(function()
192         {
193             player2.src = "imgsReflexo/macacoBase.png";
194             player1.src = "imgsReflexo/macacoBase.png";
195         }, 600);
196
197         player1.classList.add("atacando");
198     }
199 }
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224

```

Figura 3: Trecho da implementação em JavaScript do jogo Duelo de Reflexo.

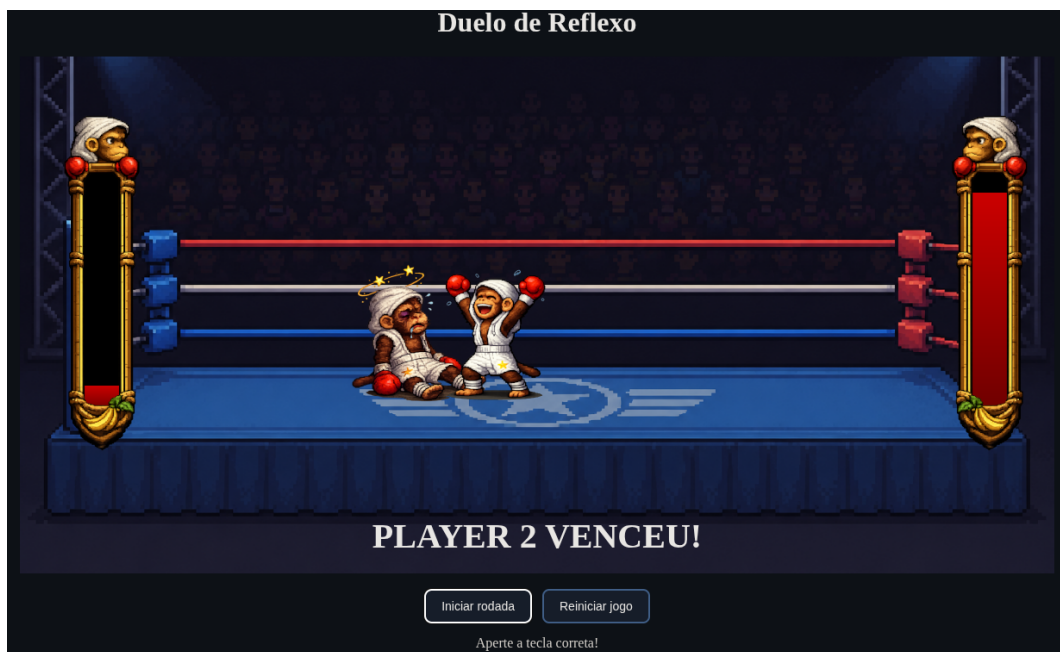


Figura 4: Tela do jogo Duelo de Reflexo em funcionamento.

1.2 Desenvolvimento do Jogo da Memória

O jogo da memória foi desenvolvido utilizando HTML, CSS e JavaScript. O objetivo do jogador é encontrar todos os pares de cartas com imagens correspondentes. Para isso, duas cartas podem ser reveladas a cada jogada.

A estrutura HTML foi utilizada para criar o tabuleiro e organizar individualmente as cartas. Cada carta possui uma face frontal, contendo uma imagem, e uma face traseira, exibida enquanto a carta ainda não foi selecionada.

```
1 <!DOCTYPE html>
2 <html lang="pt-BR">
3
4 <head>
5   <meta charset="UTF-8">
6   <title>Jogo da memória</title>
7   <link rel="stylesheet" href="Memoria.css">
8 </head>
9 <body>
10  <a href=".." index.html" class="back-link"> MENU</a>
11
12  <h1>JOGO DA MEMORIA</h1>
13  <section class="memory-game">
14    <div class="memory-card" data-framework="m1">
15      
16      
17    </div>
18
19    <div class="memory-card" data-framework="m1">
20      
21      
22    </div>
23
24    <div class="memory-card" data-framework="m2">
25      
26      
27    </div>
28
29    <div class="memory-card" data-framework="m2">
30      
31      
32    </div>
33
34    <div class="memory-card" data-framework="m3">
35      
36      
37    </div>
38
39    <div class="memory-card" data-framework="m3">
40      
41      
42    </div>
43
44    <div class="memory-card" data-framework="b1">
45      
46      
47    </div>
48
49    <div class="memory-card" data-framework="b1">
50      
51      
52    </div>
```

Figura 5: Estrutura HTML utilizada na construção do jogo da memória.

A estilização foi realizada utilizando CSS. Foram definidos o posicionamento das cartas, as dimensões do tabuleiro, a aparência das faces e o efeito de rotação utilizado para representar visualmente a abertura e o fechamento de cada carta.

A lógica da partida foi implementada em JavaScript por meio da manipulação do DOM e do tratamento de eventos de clique. Quando a primeira carta é selecionada, ela é armazenada temporariamente. Após a seleção da segunda carta, o programa compara os identificadores associados às duas imagens.

Quando as cartas formam um par, seus eventos de clique são removidos e elas permanecem abertas. Caso sejam diferentes, o tabuleiro é temporariamente bloqueado e as cartas são fechadas novamente após um pequeno intervalo de tempo. Esse bloqueio impede que outras cartas sejam selecionadas durante a animação.

As cartas são embaralhadas automaticamente no início da partida, alterando sua posição no tabuleiro. A Figura 8 apresenta o resultado visual do jogo em execução.

```

1  @font-face {
2    font-family: 'Arcade';
3    src: url('fontes/PressStart2P.ttf') format('truetype'),
4         url('fontes/PressStart2P.woff') format('woff');
5    font-weight: normal;
6    font-style: normal;
7  }
8
9  * {
10   padding: 0;
11   margin: 0;
12   box-sizing: border-box;
13 }
14
15 body {
16   justify-content: center;
17   align-items: center;
18   height: 100vh;
19   display: flex;
20   flex-direction: column;
21   background: #000000;
22   font-family: 'Arcade', sans-serif;
23 }
24
25 .memory-game {
26   width: 600px;
27   height: 600px;
28   margin: auto;
29   display: flex;
30   flex-wrap: wrap;
31   perspective: 1000px;
32 }
33
34 .memory-card {
35   width: calc(25% - 10px);
36   height: calc(25% - 10px);
37   margin: 5px;
38   position: relative;
39   box-shadow: 1px 1px 1px #000;
40   transform: scale(1);
41   transform-style: preserve-3d;
42   transition: transform .5s;
43 }
44
45 .memory-card:active {
46   transform: scale(0.97);
47   transition: transform .2s;
48 }
49

```

Figura 6: Trecho da estilização CSS utilizada no jogo da memória.

```

7  function flipCard() {
8    if (lockBoard) return;
9    if (this === firstCard) return;
10   this.classList.add('flip');
11   if (!hasFlippedCard) {
12     hasFlippedCard = true;
13     firstCard = this;
14     return;
15   }
16   secondCard = this;
17   checkForMatch();
18 }
19
20 function checkForMatch() {
21   if (firstCard.dataset.framework === secondCard.dataset.framework) {
22     disableCards();
23     return;
24   }
25   unflipCards();
26 }
27
28 function disableCards() {
29   firstCard.removeEventListener('click', flipCard);
30   secondCard.removeEventListener('click', flipCard);
31   resetBoard();
32 }
33
34 function unflipCards() {
35   lockBoard = true;
36   setTimeout(() => {
37     firstCard.classList.remove('flip');
38     secondCard.classList.remove('flip');
39     resetBoard();
40   }, 1500);
41 }
42
43 function resetBoard() {
44   [hasFlippedCard, lockBoard] = [false, false];
45   [firstCard, secondCard] = [null, null];
46 }
47
48 (function shuffle() {
49   cards.forEach(card => {
50     let randomPos = Math.floor(Math.random() * 12);
51     card.style.order = randomPos;
52   });
53 })();
54

```

Figura 7: Trecho da implementação em JavaScript do jogo da memória.



Figura 8: Tela do jogo da memória em funcionamento.

1.3 Desenvolvimento do Jogo da Velha

O jogo da velha foi desenvolvido utilizando HTML, CSS e JavaScript, oferecendo dois modos de jogo: jogador contra jogador e jogador contra computador. O objetivo foi implementar uma versão digital do jogo clássico, com controle automático das jogadas, identificação das condições de vitória e contabilização do placar entre as partidas.

A estrutura HTML foi utilizada para organizar o tabuleiro, os painéis de pontuação e os botões responsáveis pelo controle do jogo.

```
1 <link rel="stylesheet" href="style.css">
2 <html>
3 <head>
4 <div class="wrap">
5 <div class="scoreboard">
6 <div class="score-col">
7 <div class="score-lbl"> Jogador 1 </div>
8 <div class="score-num" id="s1">0</div>
9 </div>
10 <div class="score-col">
11 <div class="score-lbl"> Jogador 2 </div>
12 <div class="score-num" id="s2">0</div>
13 </div>
14 </div>
15 <div class="turn-banner pl" id="banner"> Jogador 1's turn</div>
16 </div>
17 <div class="board">
18 <div class="cell" data-i="0" onclick="move(0)"></div>
19 <div class="cell" data-i="1" onclick="move(1)"></div>
20 <div class="cell" data-i="2" onclick="move(2)"></div>
21 <div class="cell" data-i="3" onclick="move(3)"></div>
22 <div class="cell" data-i="4" onclick="move(4)"></div>
23 <div class="cell" data-i="5" onclick="move(5)"></div>
24 <div class="cell" data-i="6" onclick="move(6)"></div>
25 <div class="cell" data-i="7" onclick="move(7)"></div>
26 <div class="cell" data-i="8" onclick="move(8)"></div>
27 </div>
28 <div class="mode-row">
29 <button class="mode-btn active" id="btn0" onclick="setMode(0)"> Jogador vs Jogador </button>
30 <button class="mode-btn" id="btn1" onclick="setMode(1)"> Jogador vs CPU </button>
31 </div>
32 <button class="new-btn" onclick="newGame()"> Novo Jogo </button>
33 </div>
34 <div class="history-row" id="hist"></div>
35 </div>
36 <script src="script.js"></script>
37 </html>
```

Figura 9: Estrutura HTML utilizada na construção do jogo da velha.

A estilização foi desenvolvida utilizando CSS, sendo responsável pela organização do tabuleiro, escolha das cores, efeitos visuais e adaptação dos elementos gráficos ao tema do projeto.

```
1 @import url('https://fonts.googleapis.com/css2?family=PressStart2P&display=swap');
2 * { box-sizing: border-box; margin: 0; padding: 0; }
3 .wrap {
4   background: #1a1a1a;
5   padding: 20px 10px 30px;
6   display: flex;
7   flex-direction: column;
8   align-items: center;
9   gap: 10px;
10  font-family: 'Press Start 2P', monospace;
11 }
12 .title {
13   color: #00ffcc;
14   font-size: 1.2em;
15   letter-spacing: 0.2em;
16   text-shadow: 0 0 10px #00ffcc;
17   animation: flicker 2s linear infinite;
18 }
19 @keyframes flicker { 0%, 99%, 100% { opacity: 1; } 99%, 100% { opacity: 0.5; } }
20 .scoreboard {
21   display: flex;
22   gap: 10px;
23   border: 1px solid #00ffcc;
24   border-radius: 4px;
25   overflow: hidden;
26   width: 100%;
27   max-width: 300px;
28 }
29 .score-col {
30   flex: 1;
31   text-align: center;
32   padding: 5px 10px;
33   border-right: 1px solid #00ffcc;
34 }
35 .score-col:last-child { border-right: none; }
36 .score-lbl { color: #00ffcc; font-size: 0.8em; margin-bottom: 5px; }
37 .score-num { color: #00ffcc; font-size: 1.2em; }
38 .turn-banner {
39   font-size: 0.8em;
40   color: #00ffcc;
41   min-height: 1.2em;
42   text-align: center;
43   letter-spacing: 0.1em;
44 }
45 .cell {
46   width: 30px; height: 30px; display: flex; align-items: center; justify-content: center; font-size: 1.5em; border: 1px solid #00ffcc; border-radius: 4px; margin: 2px; background-color: #2a2a2a; }
```

Figura 10: Trecho da estilização CSS utilizada no jogo da velha.

Toda a lógica da partida foi implementada em JavaScript. Foram desenvolvidas funções para controle das jogadas, alternância entre os jogadores, verificação automática das combinações vencedoras, detecção de empate, atualização do placar e reinicialização das partidas.

A Figura abaixo apresenta o resultado final do jogo em funcionamento.

```

1 const WINES = [[0,1,2],[3,4,5],[6,7,8],[9,3,6],[2,6,7],[0,4,8],[2,6,8]];
2 const PARAS = { '▲': '1', '🐵': '2' };
3 let board, turn, over, mode = '2p', scores = [0,0,0], history = [];
4
5 function cells() { return document.querySelectorAll('cell'); }
6
7 function newGame() {
8   board = Array(9).fill(null);
9   turn = 0;
10  over = false;
11  cells().forEach(c => { c.textContent = ''; c.className = 'cell'; });
12  setBanner();
13 }
14
15 function setBanner() {
16   const b = document.getElementById('banner');
17   b.className = 'turn-banner' + (turn === 0 ? 'pl' : 'p2');
18   b.textContent = "vez do jogador " + (turn === 0 ? '1 ▲' : (mode === 'cpu' ? 'cpu 🐵' : '2 🐵'));
19 }
20
21 function setMode() {
22   mode = '1';
23   document.getElementById('btn2p').className = 'mode-btn' + (mode === '2p' ? ' active' : '');
24   document.getElementById('btncpu').className = 'mode-btn' + (mode === 'cpu' ? ' active' : '');
25   newGame();
26 }
27
28 function move(i) {
29   if (over || board[i] !== null) return;
30   place(i, turn);
31   const w = checkWin();
32   if (w) { endGame(w, turn); return; }
33   if (board.every(v => v !== null)) { endGame(null, -1); return; }
34   turn = 1 - turn;
35   setBanner();
36   if (mode === 'cpu' && turn === 1 && !over) setTimeout(cpuMove, 320);
37 }
38
39 function place(i, t) {
40   board[i] = t;
41   const c = document.querySelector('cell[data-i="' + i + '"]');
42   c.textContent = PARAS[t];
43   c.classList.add('taken');
44 }
45
46 function checkWin() {

```

Figura 11: Trecho da implementação em JavaScript do jogo da velha.



Figura 12: Tela do jogo da velha em execução.

1.4 Desenvolvimento do Jogo Pedra, Papel e Tesoura

O jogo Pedra, Papel e Tesoura foi desenvolvido utilizando HTML, CSS e JavaScript e permite partidas entre dois jogadores. Durante cada rodada, ambos realizam suas escolhas de forma independente e, após a confirmação das jogadas, o sistema compara automaticamente as opções selecionadas para determinar o vencedor.

A estrutura HTML foi utilizada para organizar as telas de escolha dos jogadores, área de exibição do resultado e controles de reinicialização da partida.

```
<div class="menu-bg-grid" aria-hidden="true"></div>

<header class="game-header">
  <a href="../../index.html" class="back-link" aria-label="Voltar ao menu"> Menu</a>
  <span class="game-header__title">Pedra, Papel e Tesoura</span>
</header>

<main id="gameStage" aria-live="polite">

  <!-- 1. Tela inicial -->
  <section class="screen screen--start active" id="screenStart">
    <p class="screen_eyebrow">Play UUEL</p>
    <h2 class="screen_title">Pedra, Papel e Tesoura</h2>
    <button class="btn-primary" id="btnStart" onclick="startGame()">Iniciar</button>
  </section>

  <!-- 2. Anúncio do jogador -->
  <section class="screen screen--announce" id="screenAnnounce">
    <div class="announce-badge" id="announceBadge">
      <span class="announce-badge_label" id="announceLabel">PLAYER 1</span>
    </div>
  </section>

  <!-- 3. Tela de escolha -->
  <section class="screen screen--choices" id="screenChoices">
    <p class="current-player-label" id="currentPlayerLabel">PLAYER 1</p>
    <p class="choice-prompt">Escolha sua jogada</p>
    <div class="choice-grid">
      <button class="choice-btn" onclick="handleChoice('rock')">
        <span class="choice-btn_emoji">🪨</span>
        <span class="choice-btn_name">Pedra</span>
      </button>
      <button class="choice-btn" onclick="handleChoice('paper')">
        <span class="choice-btn_emoji">📄</span>
        <span class="choice-btn_name">Papel</span>
      </button>
      <button class="choice-btn" onclick="handleChoice('scissors')">
        <span class="choice-btn_emoji">✂️</span>
        <span class="choice-btn_name">Tesoura</span>
      </button>
    </div>
  </section>
```

Figura 13: Estrutura HTML utilizada no jogo Pedra, Papel e Tesoura.

A estilização visual foi desenvolvida utilizando CSS, sendo responsável pelos efeitos de transição entre as telas, animações das cartas de escolha e organização dos elementos gráficos presentes na interface.

A lógica do jogo foi implementada em JavaScript, contemplando o gerenciamento dos turnos, armazenamento das escolhas dos jogadores, comparação das jogadas segundo as regras tradicionais do jogo e exibição automática do vencedor de cada rodada.

A Figura abaixo apresenta uma partida em execução.

```

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

:root {
  --bg: #0d0d14;
  --bg-surface: #16162a;
  --bg-surface-2: #1e1e38;
  --border: rgba(255,255,255,.08);
  --accent: #7c3aed;
  --accent-bright: #a855f7;
  --accent-glow: rgba(168,85,247,.35);
  --p1: #f472b6;
  --p1-glow: rgba(244,114,182,.3);
  --p2: #38bdf8;
  --p2-glow: rgba(56,189,248,.3);
  --radius-xl: 1.5rem;
  --ease: .22s cubic-bezier(.4,0,.2,1);
}

body {
  font-family: 'Segoe UI', system-ui, sans-serif;
  background: var(--bg);
  color: #fff;
  display: flex;
  flex-direction: column;
  min-height: 100dvh;
  overflow-x: hidden;
}

/* — Fundo decorativo — */
.menu-bg-grid {
  position: fixed;
  inset: 0;
  z-index: 0;
  pointer-events: none;
  background-image:
    linear-gradient(rgba(124,58,237,.07) 1px, transparent 1px),
    linear-gradient(90deg, rgba(124,58,237,.07) 1px, transparent 1px);
  background-size: 48px 48px;
  mask-image: radial-gradient(ellipse 80% 70% at 50% 50%, black 40%, transparent 100%);
}

```

Figura 14: Trecho da estilização CSS utilizada no jogo Pedra, Papel e Tesoura.

```

function showChoiceButtons(player) {
  // Update the small "PLAYER X" label at the top
  currentPlayerLabel.textContent = `PLAYER ${player}`;
  currentPlayerLabel.className = 'current-player-label';
  currentPlayerLabel.classList.add(player === 1 ? 'p1-color' : 'p2-color');

  // Re-enable all choice buttons (disabled after selection)
  document.querySelectorAll('.choice-btn').forEach(btn => {
    btn.disabled = false;
    btn.style.opacity = '1';
  });

  showScreen('choices');
}

/* — Input handling — */
/**
 * handleChoice(choice)
 * Stores the current player's choice and advances the game.
 * Buttons are disabled immediately to prevent double-clicks.
 */
function handleChoice(choice) {
  // Disable buttons immediately
  document.querySelectorAll('.choice-btn').forEach(btn => {
    btn.disabled = true;
    btn.style.opacity = '.4';
  });

  if (currentTurn === 1) {
    p1Choice = choice;
    // Wait briefly so the button state is visible, then show P2's turn
    setTimeout(() => showPlayerTurn(2), 280);
  } else {
    p2Choice = choice;
    setTimeout(revealChoices, 280);
  }
}

/* — Step 4 — Reveal choices side by side — */
/**
 * revealChoices()
 * Populates the two reveal cards and shows the reveal screen.
 * After 3 seconds, advances to showWinner().
 */
function revealChoices() {

```

Figura 15: Trecho da implementação em JavaScript do jogo Pedra, Papel e Tesoura.

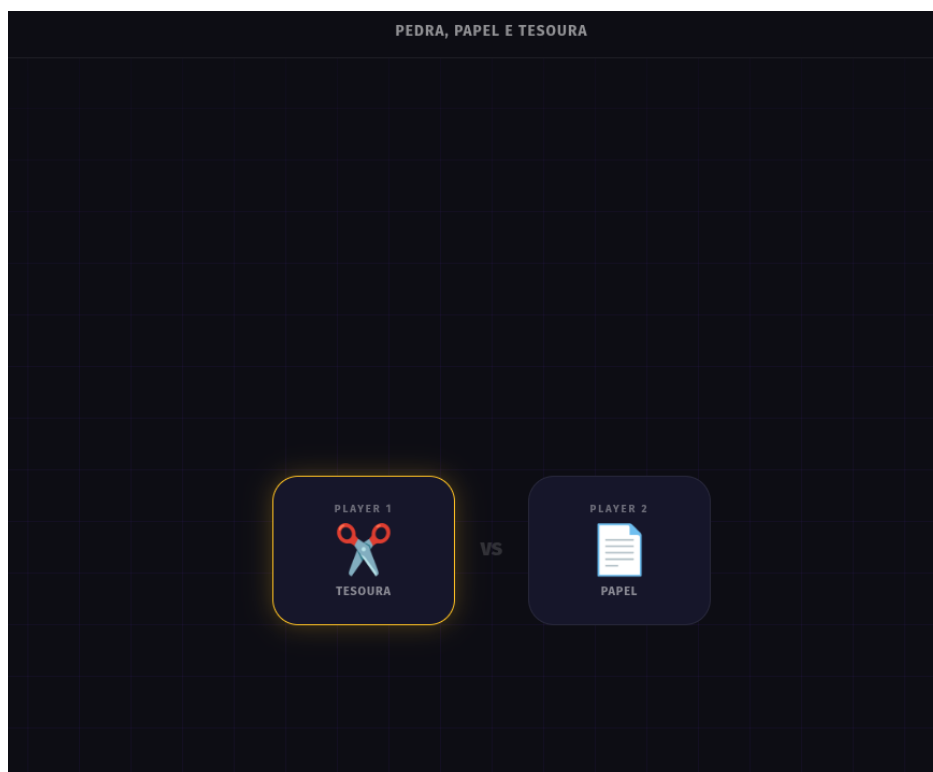


Figura 16: Tela do jogo Pedra, Papel e Tesoura em funcionamento.

1.5 Desenvolvimento do Jogo da Forca

O jogo da forca foi desenvolvido utilizando HTML, CSS e JavaScript, oferecendo os modos de um e dois jogadores. No modo individual, as palavras são selecionadas automaticamente a partir de um banco de dados interno, enquanto no modo para dois jogadores um participante informa a palavra que deverá ser descoberta pelo adversário.

A estrutura HTML organiza os elementos da interface, incluindo área de exibição da palavra, teclado virtual, campo para tentativa de resposta, painel de dicas e botões de controle da partida.

```
<html lang="pt-BR">
<body>
  <div id="overlay-vitoria">
    <div class="vitoria-titulo">PARABENS,<DF>VULE FUI<DF>NOTA UEL!</div>
    <div class="vitoria-sub">🐼 O MACACO AGRADECE! 🐼</div>
    <button class="btn-footer" id="btn-novo-vitoria" style="margin-top:8px"> JOGAR DE NOVO</button>
  </div>

  <div id="game-container">
    <h1>🐼 JOGO DA FORCA 🐼</h1>

    <!-- ===== TELA 1: ESCOLHA DE MODO ===== -->
    <div id="tela-modo">
      <p class="modo-titulo">ESCOLHA O MODO DE JOGO</p>
      <div class="modo-botoes">
        <button class="btn-modo" id="btn-1p">
          <span class="modo-icon">👤</span>
          1 JOGADOR<br><span style="font-size:7px;color:■#90e0ef">Palavra aleatória</span>
        </button>
        <button class="btn-modo" id="btn-2p">
          <span class="modo-icon">👤👤</span>
          2 JOGADORES<br><span style="font-size:7px;color:■#90e0ef">Um digita, outro adivinha</span>
        </button>
      </div>
    </div>

    <!-- ===== TELA 2: ENTRADA DE PALAVRA (2P) ===== -->
    <div id="tela-entrada">
      <p class="entrada-titulo">👤 JOGADOR 1 – DIGITE A PALAVRA</p>
      <p class="entrada-sub">👤 JOGADOR 2 NÃO PODE OLHAR!</p>
      <input
        type="password"
        id="input-palavra-2p"
        class="input-pixel"
        maxLength="20"
        placeholder="DIGITE A PALAVRA..."
        autocomplete="off"
      />
      <input
        type="text"
        id="input-dica-2p"
        class="input-pixel"
        maxLength="40"
        placeholder="DICA (OPCIONAL)..."
        autocomplete="off"
      />
      <p class="entrada-aviso" id="aviso-entrada"></p>
      <div class="btns-footer">
        <button class="btn-footer" id="btn-confirmar-palavra">CONFIRMAR</button>
      </div>
    </div>
  </div>
</body>
</html>
```

Figura 17: Estrutura HTML utilizada no jogo da forca.

A aparência visual foi desenvolvida utilizando CSS, sendo responsável pela disposição dos elementos na tela, efeitos de destaque para as letras selecionadas, animações e identidade visual baseada no tema do projeto.

Toda a lógica da aplicação foi implementada em JavaScript. Entre as funcionalidades desenvolvidas destacam-se o gerenciamento do banco de palavras, validação das letras escolhidas, atualização dinâmica da palavra exibida, controle do número de erros, sistema de dicas, verificação das condições de vitória ou derrota e reinicialização automática das partidas.

A Figura abaixo apresenta o resultado final do jogo em funcionamento.

```

@import url('https://fonts.googleapis.com/css2?family=Press+Start+2P&display=swap');

/* ===== RESET E BASE ===== */
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  background: #1a8a2e;
  min-height: 100vh;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  font-family: 'Press Start 2P', monospace;
  padding: 20px;
  overflow-x: hidden;
}

/* estrelas de fundo */
body:before {
  content: '';
  position: fixed;
  inset: 0;
  background-image:
    radial-gradient(1px 1px at 10% 15%, #fff8 0%, transparent 100%),
    radial-gradient(1px 1px at 30% 40%, #fff6 0%, transparent 100%),
    radial-gradient(1px 1px at 60% 10%, #fff8 0%, transparent 100%),
    radial-gradient(1px 1px at 80% 35%, #fff5 0%, transparent 100%),
    radial-gradient(1px 1px at 45% 75%, #fff7 0%, transparent 100%),
    radial-gradient(1px 1px at 90% 30%, #fff6 0%, transparent 100%),
    radial-gradient(1px 1px at 20% 90%, #fff8 0%, transparent 100%),
    radial-gradient(2px 2px at 70% 80%, #fff4 0%, transparent 100%);
  pointer-events: none;
  z-index: 0;
}

/* ===== CONTAINER PRINCIPAL ===== */
#game-container {
  position: relative;
  z-index: 1;
  width: 100%;
  max-width: 700px;
  background: #128b2a;
  border: 4px solid #7b2ff6;
  padding: 24px;
  box-shadow: 0 0 40px #7b2ff6, inset 0 0 30px #1a8a2e;
}

```

Figura 18: Trecho da estilização CSS utilizada no jogo da forca.

```

// =====
// CHUTAR LETRA
// =====
function chutar(letra) {
  if (!jogoAtivo || letrasChutadas.includes(letra)) return;

  letrasChutadas.push(letra);

  if (!palavraAtual.includes(letra)) {
    erros++;
    animarErro();
  }

  atualizarTeclado();
  renderPalavra();
  renderMonkey();
  atualizarVidas();
  verificarFimDeJogo();
}

// =====
// ADIVINHAR A PALAVRA INTEIRA
// =====
function tentarAdivinharPalavra() {
  if (!jogoAtivo) return;

  const input = document.getElementById('input-advinhar');
  const tentativa = input.value.trim().toUpperCase()
    .normalize('NFD').replace(/[\u0300-\u036f]/g, ''); // ignora acentos
  const alvo = palavraAtual.normalize('NFD').replace(/[\u0300-\u036f]/g, '');

  input.value = '';
  if (tentativa.length === 0) return;

  if (tentativa === alvo) {
    // Revela todas as letras
    const letrasUnicas = [...new Set(palavraAtual.split('').filter(l => l !== ' '))];
    letrasUnicas.forEach(l => {
      if (!letrasChutadas.includes(l)) letrasChutadas.push(l);
    });
    renderPalavra();
    atualizarTeclado();
    jogoAtivo = false;
    bloquearTudo();
    setTimeout(celebrarVitoria, 400);
  } else {
    erros++;
    animarErro();
  }
}

```

Figura 19: Trecho da implementação em JavaScript do jogo da forca.



Figura 20: Tela do jogo da forca em execução.

2 Resultados Obtidos

Ao término do projeto foi desenvolvida uma plataforma denominada *Play UEL*, composta por cinco jogos digitais implementados integralmente utilizando HTML, CSS e JavaScript. Todos os jogos foram integrados em uma interface única, permitindo ao usuário navegar facilmente entre as diferentes atividades propostas.

Cada jogo apresenta características próprias de interação, exigindo a implementação de diferentes estruturas de dados, manipulação de eventos, atualização dinâmica da interface e controle do estado da aplicação. Durante o desenvolvimento foram aplicados conceitos relacionados à programação web, manipulação do DOM, estilização responsiva, lógica de programação e organização modular do código.

Os testes realizados demonstraram o correto funcionamento das funcionalidades implementadas, incluindo detecção de vitórias, derrotas e empates, atualização automática das interfaces, reinicialização das partidas e interação entre jogadores. O projeto atingiu os objetivos propostos, resultando em uma plataforma funcional, visualmente padronizada e de fácil utilização.



Figura 21: Interface final da plataforma Play UEL contendo todos os jogos desenvolvidos.